

Linux 驱动开发基础面试题

本文档由公众号：**嵌入式校招菌** 原创整理，欢迎关注获取更多嵌入式校招信息

🔴 **嵌入式校招excel** 全年更新中，实习/秋招/春招都包含，纯人工维护，已支持24/25/26届同学毕业，减少招聘信息差，你没听过的公司只要有嵌入式岗位我都会收集，一次付费永久有效，更有嵌入式微信/飞书交流群；用过的同学都说好！

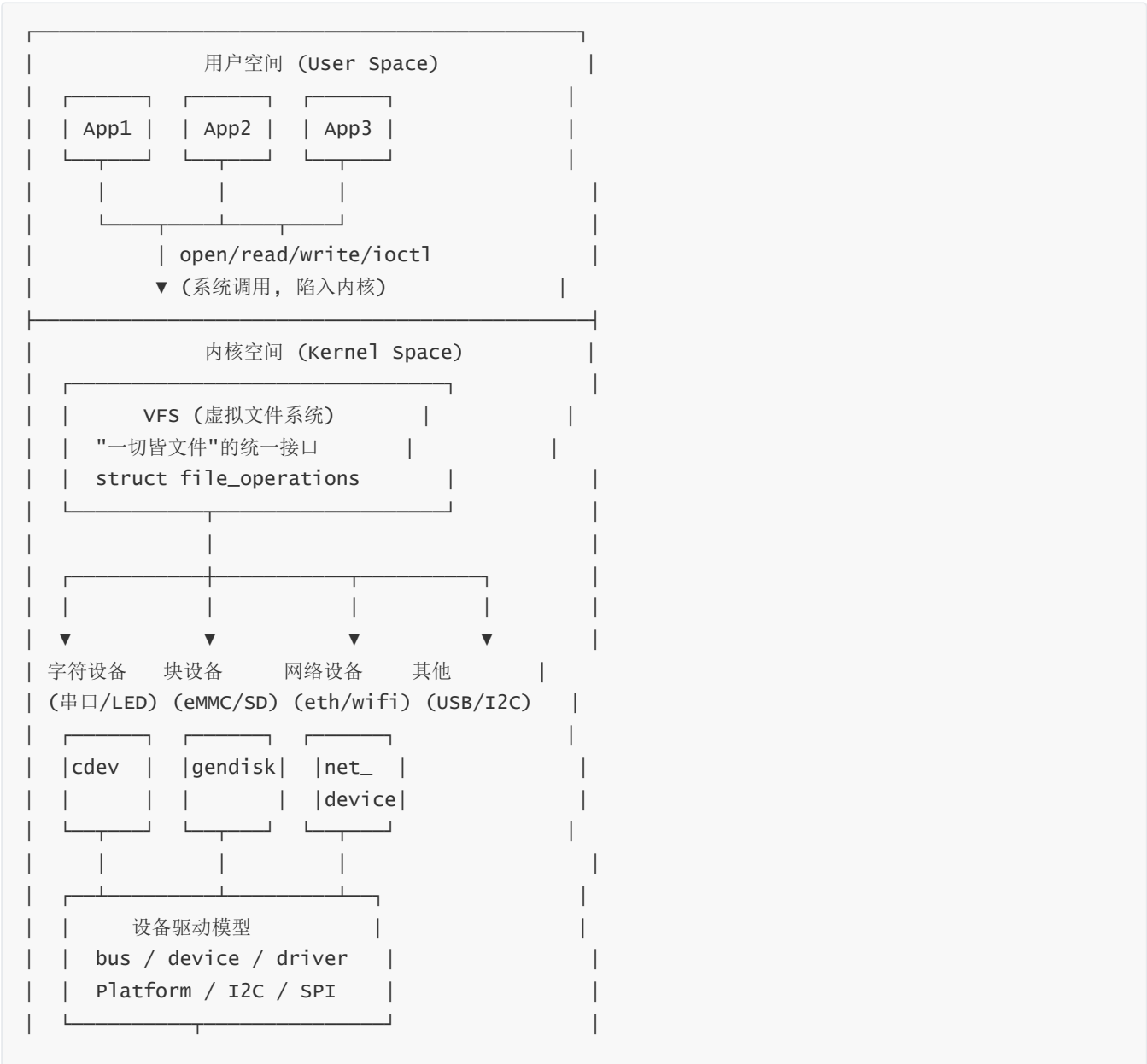
需要的可以关注公众号：**嵌入式校招菌**，对话框回复：**加群**

精选 155 道 Linux 内核驱动高频面试题，涵盖字符设备、平台模型、设备树、中断、内存管理、并发同步、子系统等。

每题配详细答案、代码示例和架构图。

★ Linux驱动核心概念图解（先理解架构，再刷面试题）

◆ Linux驱动在操作系统中的位置



硬件

GPIO / UART / SPI / I2C / DMA / 中断

◆ 字符设备驱动框架

```
// 字符设备驱动最小模板(必须背!)
#include <linux/module.h>
#include <linux/fs.h>
#include <linux/cdev.h>

static struct cdev my_cdev;
static dev_t dev_num;

// ① 定义 file_operations (连接用户空间和驱动)
static int my_open(struct inode *inode, struct file *file) { return 0; }
static ssize_t my_read(struct file *f, char __user *buf, size_t len, loff_t *off) {
    copy_to_user(buf, kernel_buf, len); // 内核→用户(必须用copy_to_user)
    return len;
}
static struct file_operations my_fops = {
    .owner = THIS_MODULE,
    .open = my_open,
    .read = my_read,
};

// ② 模块加载
static int __init my_init(void) {
    alloc_chrdev_region(&dev_num, 0, 1, "mydev"); // 动态分配设备号
    cdev_init(&my_cdev, &my_fops); // 绑定fops
    cdev_add(&my_cdev, dev_num, 1); // 注册到内核
    // class_create + device_create → 自动创建/dev节点
    return 0;
}

// ③ 模块卸载
static void __exit my_exit(void) {
    cdev_del(&my_cdev);
    unregister_chrdev_region(dev_num, 1);
}
module_init(my_init);
module_exit(my_exit);
MODULE_LICENSE("GPL");
```

◆ Platform驱动模型与设备树

为什么需要Platform模型？

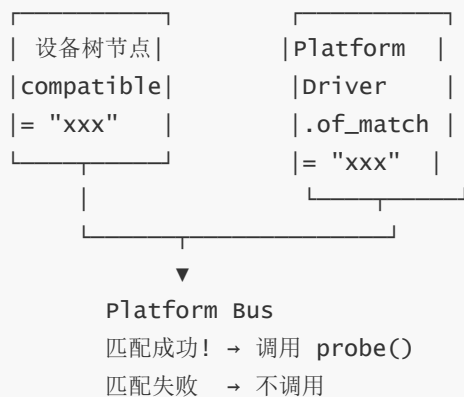
- 将"硬件信息"和"驱动逻辑"分离(解耦)
- 硬件信息放设备树(.dts)，驱动逻辑放.c文件
- 换一块板子只改设备树,驱动代码不用动！

设备树(.dts) → 编译 → .dtb → U-Boot传给内核

设备树示例：

```
my_led {  
    compatible = "mycompany,led"; ← ★匹配关键字  
    reg = <0x40020000 0x400>;      ← 寄存器基地址  
    gpios = <&gpioa 5 GPIO_ACTIVE_HIGH>;  
};
```

驱动匹配流程：

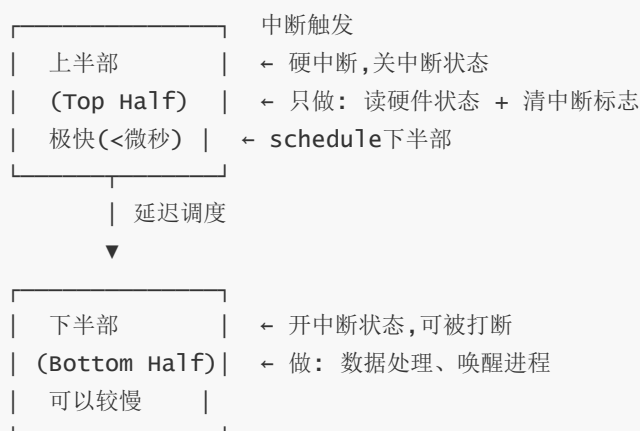


- ★ probe()时才真正初始化硬件(申请资源/注册设备)
- ★ remove()时释放资源

◆ 中断处理:上半部 vs 下半部

为什么要分两半？

- 中断处理期间会屏蔽同级中断 → 必须极快退出！
- 耗时操作(如读I2C传感器数据)不能在中断里做



下半部实现方式：

	softirq	tasklet	workqueue
上下文	软中断	软中断	进程(可睡眠)
能睡眠?	不能	不能	能 ★
并发	可多CPU	同类不并	可多CPU
典型用途	网络/块	简单延迟	I2C/SPI读写

★ 需要睡眠(如I2C读传感器) → 必须用 workqueue

★ 不需要睡眠的简单工作 → tasklet

◆ 内核同步机制

机制	可睡眠?	适用上下文	典型场景
spin_lock	不可	中断/进程	短临界区
mutex	可	仅进程	长临界区
semaphore	可	仅进程	计数资源
rw_lock	不可	中断/进程	读多写少(短)
RCU	读不可	中断/进程	读多写极少
atomic_t	不可	任何	简单计数
completion	可	仅进程	等待事件完成

★ 中断上下文(上半部/softirq/tasklet)不能睡眠!

→ 不能用mutex/semaphore, 只能用spin_lock

★ spin_lock在单CPU上退化为关抢占(不真的自旋)

★ 中断中访问共享资源 → spin_lock_irqsave()

一、内核模块基础 (Q1~Q19)

Q1: Linux 内核模块的基本结构?

💡 秒懂： 内核模块的基本框架：#include头文件→module_init()/module_exit()注册初始化/退出函数
→MODULE_LICENSE声明许可证。就像一个有'入口'和'出口'的插件。

关键点: Linux驱动分三类: 字符设备(串口/LED,顺序访问)、块设备(磁盘,随机访问)、网络设备(网卡,socket接口)。面试最常考字符设备驱动。

```
#include <linux/init.h>
#include <linux/module.h>

static int __init my_init(void) {
    printk(KERN_INFO "Hello kernel!\n");
    return 0; /* 0 = 成功, 非零 = 失败 */
}
```

```
static void __exit my_exit(void) {
    printk(KERN_INFO "Bye kernel!\n");
}

module_init(my_init);    /* 指定入口函数 */
module_exit(my_exit);    /* 指定出口函数 */
MODULE_LICENSE("GPL");    /* 必须声明许可证 */
MODULE_AUTHOR("xxx");
MODULE_DESCRIPTION("demo module");
```

- `__init`: 初始化完成后释放该函数内存
- `__exit`: 仅卸载时用到, 编译进内核时不生成

Linux三大设备类型对比:

特性	字符设备	块设备	网络设备
访问方式	字节流,顺序访问	固定大小块,随机访问	数据包,协议栈驱动
缓冲	无系统缓冲	有缓冲区(page cache)	sk_buff缓冲
设备节点	/dev/ttyS0	/dev/sda	无设备节点(eth0)
核心结构	file_operations	block_device_operations	net_device_ops
注册函数	cdev_add()	register_blkdev()	register_netdev()
典型设备	串口/GPIO/传感器	eMMC/NAND/SD卡	以太网/WiFi/CAN
用户空间接口	open/read/write/ioctl	mount后通过VFS	socket API

Q2: insmod/rmmod/modprobe 区别?

 **秒懂:** insmod直接加载(不解决依赖), modprobe自动加载依赖模块(推荐), rmmod卸载模块。lsmod查看已加载模块。开发时insmod方便, 部署时用modprobe。

```
insmod my_driver.ko    # 直接加载, 不处理依赖
rmmod my_driver        # 直接卸载
modprobe my_driver      # 自动处理依赖关系
modprobe -r my_driver   # 卸载并处理依赖
lsmod                  # 查看已加载模块
modinfo my_driver.ko    # 查看模块信息
```

modprobe 需要模块在 `/lib/modules/$(uname -r)/` 目录下并运行过 `depmod`。

 **面试追问:** insmod/modprobe的区别? 模块依赖怎么处理? 模块参数怎么传?

 **嵌入式建议:** modprobe自动处理依赖(推荐);insmod手动指定路径。开发阶段用insmod方便;产品用modprobe。

Q3: 模块参数传递?

 **秒懂:** module_param(name, type, perm)声明模块参数→insmod xxx.ko param=value传入。参数出现在/sys/module/xxx/parameters/。用于配置驱动行为而无需重新编译。

Linux内核模块加载时可通过module_param宏接收用户传入的参数:

```
static int baud = 115200;
static char *name = "uart0";
module_param(baud, int, 0644);      /* 0644 = /sys/module/xxx/parameters/ 可读写 */
module_param(name, charp, 0444);
MODULE_PARM_DESC(baud, "Baud rate, default 115200");

/* 数组参数 */
static int arr[4];
static int arr_count;
module_param_array(arr, int, &arr_count, 0644);
```

加载时传参: `insmod my.ko baud=9600 name="uart1"`

Q4: printk 日志级别?

 **秒懂:** printk有8个级别(0-7): KERN_EMERG(0紧急)到KERN_DEBUG(7调试)。dmesg查看内核日志。嵌入式驱动开发中printk是最基本的调试手段。pr_info/pr_err是更简洁的包装。

printk是内核空间的打印函数, 通过日志级别控制输出:

KERN_EMERG	0	系统不可用
KERN_ALERT	1	需要立即动作
KERN_CRIT	2	严重条件
KERN_ERR	3	错误
KERN_WARNING	4	警告
KERN_NOTICE	5	正常但显著
KERN_INFO	6	信息
KERN_DEBUG	7	调试


```
/* 查看/设置当前打印级别 */
cat /proc/sys/kernel/printk
/* 输出: 4 4 1 7 → 当前/默认/最低控制台级别/默认控制台 */
echo 8 > /proc/sys/kernel/printk /* 允许所有级别 */
```

 **面试追问:** 字符设备/块设备/网络设备的区别? 举例子?

 **嵌入式建议:** 嵌入式自定义驱动90%是字符设备(GPIO/传感器/控制器)。块设备(Flash/SD)和网设备(ETH)通常用内核现有框架。

Linux三种设备类型对比

类型	访问方式	代表设备	设备文件	缓冲
字符设备(char)	顺序字节流	UART/GPIO/I2C/Key	/dev/ttyS0	无(直接读写)
块设备(block)	按块随机访问	eMMC/SD卡/U盘	/dev/mmcblk0	有(page cache)
网络设备(net)	socket接口	网卡(eth0/wlan0)	无设备文件	sk_buff

💡 嵌入式驱动: 90%是字符设备驱动, 掌握file_operations结构体是基础

Q5: EXPORT_SYMBOL 的作用?

🧠 秒懂: EXPORT_SYMBOL将一个符号导出供其他内核模块使用。不导出的函数/变量对其他模块不可见。EXPORT_SYMBOL_GPL只允许GPL模块使用。模块间协作的基础。

```
/* module_a.c */
int shared_func(int x) { return x * 2; }
EXPORT_SYMBOL(shared_func);          /* 所有模块可用 */
EXPORT_SYMBOL_GPL(shared_func);      /* 仅 GPL 模块可用 */

/* module_b.c */
extern int shared_func(int);          /* 声明后直接调用 */
```

导出符号会出现在 `/proc/kallsyms` 中。

💡 面试追问: 主设备号和次设备号的区别? 怎么区分不同设备? 动态分配和静态分配设备号的区别?

🔧 嵌入式建议: 新驱动一律用alloc_chrdev_region()动态分配(避免冲突)。主设备号标识驱动类型,次设备号标识具体设备。

Q6: 用户空间与内核空间?

🧠 秒懂: 内核空间运行在特权级(可访问所有硬件和内存), 用户空间运行在非特权级(受限)。两者之间通过系统调用(read/write/ioctl)和copy_to/from_user传递数据。

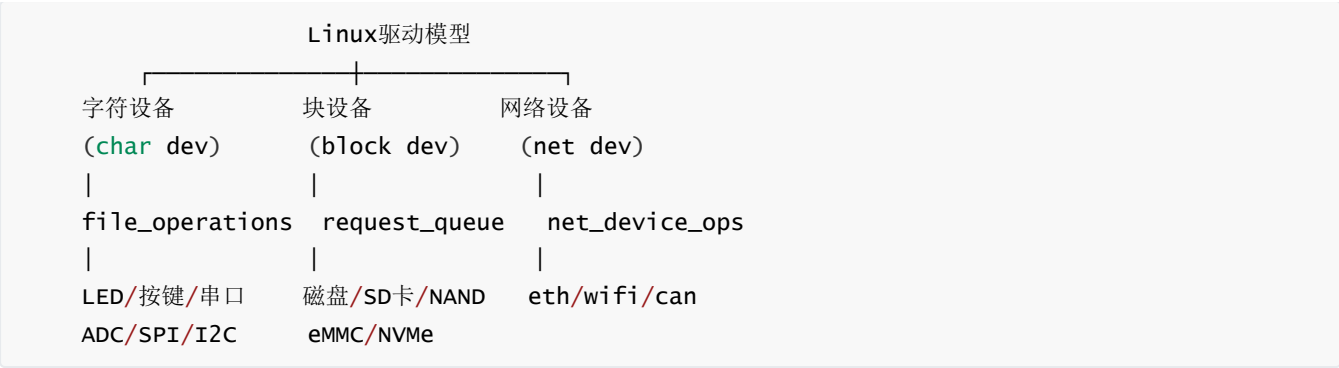
内核空间 and 用户空间的隔离是系统安全和稳定的基础:



64位系统: 用户空间 0~0x7FFFFFFFFFFFFFFF, 内核空间 0xFFFFF8000000000000 以上

用户空间不能直接访问内核空间, 必须通过系统调用(软中断)切换。

★ Linux驱动核心知识框架:



★ 驱动开发常考对比表：

机制	适用场景	能否睡眠	延迟	典型用法
硬中断(top half)	紧急/快速处理	✗ 不能	最低	清中断标志/读FIFO
tasklet	中断下半部	✗ 不能	低	网卡收包处理
workqueue	中断下半部	✓ 可以	中	I2C/SPI通信
定时器(timer)	周期/延时	✗ 不能	可配	超时检测/心跳
内核线程	后台常驻任务	✓ 可以	高	kworker/flush

同步机制	适用场景	能否睡眠	嵌入式常用度
spin_lock	中断上下文/短临界区	✗	★★★★★
mutex	进程上下文/长临界区	✓	★★★★★
semaphore	信号量计数	✓	★★★
atomic	简单计数器	不涉及	★★★★
RCU	读多写少	读不锁	★★★

Q7: 内核模块Makefile怎么写？

💡 秒懂： 内核模块用特殊的Makefile：obj-m := xxx.o，然后make -C /lib/modules/\$(uname -r)/build M=\$PWD modules。交叉编译时指定ARCH和CROSS_COMPILE。

内核模块编译使用内核构建系统(Kbuild)：

```
# 内核模块Makefile
obj-m += mymodule.o           # 单文件模块
# obj-m += mydriver.o         # 多文件模块
# mydriver-objs := file1.o file2.o

KERNELDIR ?= /lib/modules/$(shell uname -r)/build
PWD := $(shell pwd)

all:
```



```
$(MAKE) -C $(KERNELDIR) M=$(PWD) modules

clean:
    $(MAKE) -C $(KERNELDIR) M=$(PWD) clean

# 交叉编译
# KERNELDIR = /path/to/arm-kernel
# ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- make
```

Q8: 内核版本兼容性问题？

 **秒懂：** 内核API可能在不同版本间变化。用version.h中的宏检查版本(LINUX_VERSION_CODE)→条件编译适配。编写跨版本通用驱动需要做好版本兼容。

不同内核版本API可能变化，模块需要适配：

```
#include <linux/version.h>

#if LINUX_VERSION_CODE >= KERNEL_VERSION(5, 0, 0)
    // 5.0+的API
#else
    // 旧版API
#endif

// 常见变化：
// 4.x→5.x: access_ok()参数减少
// 5.6+: proc_ops替代file_operations
// 5.15+: class_create()参数减少
```

Q9: 内核内存分配(kmalloc/vmalloc/kzalloc)?

 **秒懂：** kmalloc分配连续物理内存(小块,GFP_KERNEL可睡眠/GFP_ATOMIC不睡眠)。vmalloc分配虚拟地址连续但物理可能不连续(大块)。kzalloc = kmalloc + memset零初始化。

内核中不同的内存分配方式：

函数	特点	适用场景
kmalloc	物理连续，快	小块(<128KB)，DMA
kzalloc	kmalloc+清零	结构体分配
vmalloc	虚拟连续，物理不连续	大块内存
kfree/vfree	对应释放	

```
// GFP标志(分配位置/行为)
void *p = kmalloc(1024, GFP_KERNEL); // 可睡眠(进程上下文)
void *p = kmalloc(1024, GFP_ATOMIC); // 不可睡眠(中断上下文)
void *p = kzalloc(sizeof(struct mydev), GFP_KERNEL);

// 大块(>128KB)
void *p = vmalloc(1024 * 1024); // 1MB
vfree(p);
```

Q10: 内核链表(list_head)的使用?

 **秒懂：** Linux内核不用标准链表，而是把struct list_head嵌入数据结构中。
list_add/list_del/list_for_each_entry遍历。所有内核子系统都大量使用这种'侵入式链表'。

Linux内核使用侵入式链表(结构体嵌入list_head):

```
#include <linux/list.h>

struct my_node {
    int data;
    struct list_head list; // 嵌入链表节点
};

// 声明并初始化链表头
LIST_HEAD(my_list);

// 添加节点
struct my_node *node = kzalloc(sizeof(*node), GFP_KERNEL);
node->data = 42;
list_add(&node->list, &my_list); // 头插
list_add_tail(&node->list, &my_list); // 尾插

// 遍历
struct my_node *pos;
list_for_each_entry(pos, &my_list, list) {
    printk("data = %d\n", pos->data);
}

// 安全遍历(可删除)
struct my_node *tmp;
list_for_each_entry_safe(pos, tmp, &my_list, list) {
    list_del(&pos->list);
    kfree(pos);
}
```

Q11: container_of宏的原理?

 **秒懂：** container_of通过结构体成员地址反推结构体首地址: (type)((char)ptr - offsetof(type, member))。
是内核链表、设备模型等的核心宏。理解它才能看懂内核代码。

container_of是Linux内核最重要的宏之一(由成员指针获取结构体指针):

```
#define container_of(ptr, type, member) ({\
    const typeof(((type *)0)->member) *__mptr = (ptr);\
    (type *)((char *)__mptr - offsetof(type, member));})

// 原理：成员地址 - 成员在结构体中的偏移 = 结构体首地址

// 使用场景(驱动中常见)
struct my_device {
    int id;
    struct cdev cdev; // 字符设备
};

// 在file_operations中通过inode->i_cdev获取my_device
struct my_device *dev = container_of(inode->i_cdev, struct my_device, cdev);
```

Q12: 内核中的错误处理(ERR_PTR/IS_ERR)?

 **秒懂：** 内核用ERR_PTR将错误码编码为指针(地址空间最高页)，IS_ERR检查是否是错误，PTR_ERR提取错误码。这样函数可以用指针返回值同时表示成功(有效指针)和失败(错误码)。

内核通过指针编码错误码(避免额外的错误参数)：

```
// 错误指针：将错误码编码到指针最高位
void *ptr = some_kernel_func();
if (IS_ERR(ptr)) {
    int err = PTR_ERR(ptr);
    printk("Error: %d\n", err);
    return err;
}

// 返回错误指针
struct device *create_device(...) {
    if (failed)
        return ERR_PTR(-ENOMEM);
    return dev;
}
```

Q13: 内核中的工作队列(workqueue)?

 **秒懂：** workqueue在内核线程上下文中异步执行工作——可以睡眠。INIT_WORK初始化→schedule_work调度。中断下半部处理、延时操作、需要睡眠的内核任务都用workqueue。

工作队列用于延迟执行耗时任务(可睡眠上下文)：

```
#include <linux/workqueue.h>

// 方法1：系统默认workqueue
struct work_struct my_work;

void work_handler(struct work_struct *work) {
    // 可以睡眠！可以调用kmalloc(GFP_KERNEL)
    printk("work executed\n");
}
```

```

}

INIT_WORK(&my_work, work_handler);
schedule_work(&my_work); // 提交到系统workqueue

// 方法2: 延迟工作
struct delayed_work my_delayed_work;
INIT_DELAYED_WORK(&my_delayed_work, work_handler);
schedule_delayed_work(&my_delayed_work, msecs_to_jiffies(1000));

```

Q14: 内核定时器的使用?

 **秒懂:** 内核定时器在指定时间后执行回调。timer_setup初始化→mod_timer设置到期时间→回调执行。注意: 定时器回调在软中断上下文中运行, 不能睡眠。

内核定时器用于延迟执行(软中断上下文):

```

#include <linux/timer.h>

struct timer_list my_timer;

void timer_callback(struct timer_list *t) {
    printk("Timer fired!\n");
    mod_timer(&my_timer, jiffies + HZ); // 再次触发(1秒后)
}

// 初始化
timer_setup(&my_timer, timer_callback, 0);
mod_timer(&my_timer, jiffies + HZ); // 1秒后触发

// 删除
del_timer_sync(&my_timer);

```

Q15: 内核中的延迟函数(mdelay/msleep)?

 **秒懂:** mdelay忙等(不交出CPU), msleep/usleep_range交出CPU(可睡眠)。中断上下文只能用mdelay/udelay。进程上下文优先用msleep(节省CPU)。usleep_range更精确。

不同上下文使用不同延迟方式:

函数	机制	可否睡眠	适用上下文
ndelay/udelay/mdelay	忙等	否	任何(中断/进程)
usleep_range	睡眠	是	仅进程上下文
msleep	睡眠	是	仅进程上下文
schedule_timeout	睡眠	是	仅进程上下文

```
// 中断中短延迟
udelay(10); // 10微秒(忙等)

// 进程上下文中延迟
msleep(100); // 100毫秒(睡眠,不浪费CPU)
usleep_range(1000, 1500); // 1~1.5ms(精度更好)
```

💡 **面试追问：** "中断中能用msleep吗？" → 绝对不能!msleep会睡眠(调schedule),中断上下文不能睡眠→kernel panic。中断中只能用mdelay(忙等)或udelay——但应尽量避免在ISR中做长延时,用workqueue延迟到线程上下文。

Q16: 内核中的互斥与同步机制？

💡 **秒懂：** 互斥锁(mutex,可睡眠)、自旋锁(spinlock,不可睡眠)、信号量(semaphore)、RCU(读多写少)、完成量(completion,等待事件)。中断中只能用自旋锁，进程上下文优先用mutex。

Linux内核提供多种同步原语：

机制	特点	适用场景
spinlock	忙等,不可睡眠	中断上下文/短临界区
mutex	可睡眠	进程上下文/长临界区
semaphore	计数,可睡眠	资源计数控制
rwlock	读共享写排他	读多写少
RCU	读无锁	读极多写极少
atomic	原子操作	简单计数/标志
completion	等待完成	等待事件

Q17: 完成量(completion)的使用？

💡 **秒懂：** completion用于等待某个事件完成：init_completion→一方wait_for_completion等待→另一方complete通知。比信号量更直观。常用于等待DMA完成、固件加载完成等。

completion用于一个线程等待另一个线程完成工作：

```
#include <linux/completion.h>

DECLARE_COMPLETION(my_comp);

// 等待方
wait_for_completion(&my_comp); // 阻塞直到complete
// 或带超时
wait_for_completion_timeout(&my_comp, msecs_to_jiffies(1000));

// 完成方(如中断处理/另一线程)
complete(&my_comp); // 唤醒一个等待者
complete_all(&my_comp); // 唤醒所有等待者
```

Q18: 内核中的原子操作?

 **秒懂:** atomic_t类型和atomic_add/sub/read/set等函数。保证操作的原子性(不被中断打断)。用于简单的计数器和标志位——比锁轻量得多。

原子操作用于简单的整数操作(无需加锁):

```
#include <linux/atomic.h>

atomic_t counter = ATOMIC_INIT(0);

atomic_inc(&counter); // counter++
atomic_dec(&counter); // counter--
atomic_add(5, &counter); // counter += 5
int val = atomic_read(&counter); // 读取
atomic_set(&counter, 10); // 设置

// 带返回值
int old = atomic_inc_return(&counter); // ++counter并返回新值
if (atomic_dec_and_test(&counter)) // --counter==0?
    printk("Counter reached zero\n");
```

Q19: 内核日志和调试方法?

 **秒懂:** printk(基本)→动态调试(dynamic_debug可运行时开关)→ftrace(内核函数跟踪)→KGDB(内核GDB调试)→devmem(直接读写寄存器)。从简单到复杂逐步排查问题。

Linux内核调试方法综述:

```
// printk级别
printk(KERN_EMERG    "Emergency\n");    // 0 系统崩溃
printk(KERN_ERR      "Error\n");        // 3 错误
printk(KERN_WARNING  "Warning\n");      // 4 警告
printk(KERN_INFO     "Info\n");         // 6 信息
printk(KERN_DEBUG    "Debug\n");        // 7 调试

// 推荐使用dev_xxx(带设备信息)
dev_err(&pdev->dev, "Init failed: %d\n", ret);
dev_info(&pdev->dev, "Driver loaded\n");

// 动态调试
echo "module mymod +p" > /sys/kernel/debug/dynamic_debug/control
```

二、字符设备驱动（Q20~Q32）

Q20: 字符设备驱动的框架？

 **秒懂：** 字符设备驱动框架：分配设备号→初始化cdev→注册file_operations(open/read/write/ioctl等回调)→创建设备节点(/dev/xxx)。用户空间open/read设备时内核调用对应回调。

字符设备是Linux驱动最基本的类型：

```
/* 字符设备驱动的框架？ - 示例实现 */
#include <linux/cdev.h>
#include <linux/fs.h>

static int my_open(struct inode *inode, struct file *filp) { return 0; }
static int my_release(struct inode *inode, struct file *filp) { return 0; }
static ssize_t my_read(struct file *filp, char __user *buf,
                      size_t count, loff_t *pos) {
    char kbuf[] = "hello";
    if (copy_to_user(buf, kbuf, sizeof(kbuf)))
        return -EFAULT;
    return sizeof(kbuf);
}
static ssize_t my_write(struct file *filp, const char __user *buf,
                      size_t count, loff_t *pos) {
    char kbuf[64];
    if (count > sizeof(kbuf)) return -ENOMEM;
    if (copy_from_user(kbuf, buf, count))
        return -EFAULT;
    printk("Got: %.*s\n", (int)count, kbuf);
    return count;
}

static struct file_operations my_fops = {
    .owner = THIS_MODULE,
    .open = my_open,
    .release = my_release,
    .read = my_read,
```

```
.write = my_write,  
};
```

Q21: 设备号的分配(静态/动态)?

 **秒懂**: 静态分配: register_chrdev_region(手动指定主设备号), 提前知道编号。动态分配: alloc_chrdev_region(内核分配), 推荐——避免和已有设备号冲突。

设备号由主设备号(标识驱动)和次设备号(标识设备实例)组成:

```
dev_t devno;  
  
// 动态分配(推荐)  
alloc_chrdev_region(&devno, 0, 1, "mydev");  
int major = MAJOR(devno);  
int minor = MINOR(devno);  
  
// 静态分配(指定主设备号)  
devno = MKDEV(240, 0);  
register_chrdev_region(devno, 1, "mydev");  
  
// 释放  
unregister_chrdev_region(devno, 1);
```

Q22: cdev注册和自动创建设备节点?

 **秒懂**: cdev_init绑定file_operations→cdev_add注册设备→class_create+device_create自动在/dev/下创建设备节点(不需要手动mknod)。这是现代驱动的标准做法。

现代驱动使用class+device实现自动创建/dev节点:

```
static struct cdev my_cdev;  
static struct class *my_class;  
static dev_t devno;  
  
static int __init my_init(void) {  
    alloc_chrdev_region(&devno, 0, 1, "mydev");  
  
    cdev_init(&my_cdev, &my_fops);  
    cdev_add(&my_cdev, devno, 1);  
  
    // 自动创建/dev/mydev(udev/mdev)  
    my_class = class_create(THIS_MODULE, "myclass");  
    device_create(my_class, NULL, devno, NULL, "mydev");  
  
    return 0;  
}  
  
static void __exit my_exit(void) {  
    device_destroy(my_class, devno);  
    class_destroy(my_class);  
    cdev_del(&my_cdev);  
}
```



```
unregister_chrdev_region(devno, 1);
}
```

Q23: copy_to_user/copy_from_user为什么必须使用?

 **秒懂**: 内核空间不能直接访问用户空间地址(可能无效或无映射)。copy_to_user/copy_from_user做安全检查(验证地址合法性)后才拷贝。直接用memcpy可能导致内核崩溃。

内核空间 and 用户空间不能直接互相访问(安全检查+缺页处理):

```
// 必须用copy_to/from_user的原因:
// 1. 地址合法性检查(access_ok)
// 2. 用户态地址可能未映射(触发缺页中断)
// 3. 如果直接memcpy → 内核oops崩溃

ssize_t my_read(struct file *filp, char __user *buf, size_t count, loff_t *pos) {
    char kernel_buf[64] = "data from kernel";
    int len = strlen(kernel_buf) + 1;

    if (count < len) len = count;
    if (copy_to_user(buf, kernel_buf, len))
        return -EFAULT; // 地址无效
    return len;
}
```

Q24: ioctl的实现?

 **秒懂**: ioctl实现设备控制命令: 用户空间ioctl(fd, cmd, arg)→内核unlocked_ioctl回调。cmd用IO/IOR/IOW/IOWR宏编码(类型+序号+方向+大小), 防止命令号冲突。

ioctl用于设备特定的控制命令(非read/write):

```
// 定义ioctl命令码
#define MY_IOC_MAGIC 'k'
#define MY_IOC_GET_STATUS _IOR(MY_IOC_MAGIC, 1, int)
#define MY_IOC_SET_CONFIG _IOW(MY_IOC_MAGIC, 2, struct my_config)
#define MY_IOC_RESET _IO(MY_IOC_MAGIC, 3)

static long my_ioctl(struct file *filp, unsigned int cmd, unsigned long arg) { // 设备控制操作
    switch (cmd) {
        case MY_IOC_GET_STATUS: {
            int status = get_hw_status();
            if (copy_to_user((void __user *)arg, &status, sizeof(status)))
                return -EFAULT;
            return 0;
        }
        case MY_IOC_SET_CONFIG: {
            struct my_config cfg;
            if (copy_from_user(&cfg, (void __user *)arg, sizeof(cfg)))
                return -EFAULT;
            apply_config(&cfg);
        }
    }
}
```

```

        return 0;
    }
    case MY_IOC_RESET:
        hw_reset();
        return 0;
    }
    return -ENOTTY;
}

```

Q25: poll/select在驱动中的实现?

 **秒懂**：驱动实现poll回调：初始化等待队列→poll_wait注册等待队列到poll_table→返回可读/可写/异常的掩码。当数据就绪时wake_up唤醒等待的poll/select。

实现阻塞IO和poll/select/epoll支持：

```

#include <linux/poll.h>
#include <linux/wait.h>

static wait_queue_head_t my_wq;
static int data_ready = 0;

static __poll_t my_poll(struct file *filp, poll_table *wait) {
    __poll_t mask = 0;
    poll_wait(filp, &my_wq, wait);
    if (data_ready)
        mask |= EPOLLIN | EPOLLRDNORM;
    return mask;
}

static ssize_t my_read(struct file *filp, char __user *buf,
                      size_t count, loff_t *pos) {
    // 阻塞等待数据
    wait_event_interruptible(my_wq, data_ready != 0);
    // 返回数据...
    data_ready = 0;
    return count;
}

// 在中断或数据到达时唤醒
void data_arrived(void) {
    data_ready = 1;
    wake_up_interruptible(&my_wq);
}

```

Q26: 驱动中的异步通知(fasync)?

 **秒懂**：fasync实现异步通知：用户设置FASYNC标志→驱动fasync回调注册→数据就绪时kill_fasync发送SIGIO→用户的信号处理函数被调用。比poll更主动。

fasync允许驱动向进程发信号(SIGIO)通知事件：

```
static struct fasync_struct *my_fasync;

static int my_fasync_func(int fd, struct file *filp, int on) {
    return fasync_helper(fd, filp, on, &my_fasync);
}

// 数据到达时通知
void notify_app(void) {
    if (my_fasync)
        kill_fasync(&my_fasync, SIGIO, POLL_IN);
}

static struct file_operations fops = {
    .fasync = my_fasync_func,
    // ...
};
```

- 💡 面试追问： compatible怎么匹配？ of_match_table优先级？ probe什么时候被调？
- 🔧 嵌入式建议： 现代驱动标配platform_driver+设备树。probe()里做:ioremap+request_irq+注册字符设备。

🏠 Linux设备驱动模型对比

框架	层次结构	匹配方式	典型场景
platform	bus→driver→device	设备树/名字/id_table	SoC内部外设
I2C子系统	adapter→client→driver	设备树/i2c_device_id	传感器/EEPROM/RTC
SPI子系统	master→device→driver	设备树/spi_device_id	Flash/显示屏/ADC
USB子系统	HCD→hub→device→driver	VID/PID	USB外设
input子系统	handler→device	事件类型	键盘/触摸/传感器

Q27: proc文件系统接口创建？

- 💡 秒懂： proc_create创建/proc/xxx文件→注册read/write回调(proc_ops)。/proc中的文件是虚拟的——读取时内核动态生成内容(如显示驱动状态)，写入时执行对应操作。

在/proc下创建调试接口：

```
#include <linux/proc_fs.h>
#include <linux/seq_file.h>

static int my_proc_show(struct seq_file *m, void *v) {
    seq_printf(m, "status: %d\n", device_status);
    seq_printf(m, "count: %d\n", access_count);
    return 0;
}
```

```
static int my_proc_open(struct inode *inode, struct file *file) {
    return single_open(file, my_proc_show, NULL);
}

static const struct proc_ops my_proc_ops = {
    .proc_open = my_proc_open,
    .proc_read = seq_read,
    .proc_lseek = seq_lseek,
    .proc_release = single_release,
};

// 创建/proc/mydriver
proc_create("mydriver", 0444, NULL, &my_proc_ops);
```

Q28: sysfs属性文件创建?

 **秒懂**: device_create_file或DEVICE_ATTR宏创建/sys/devices/.../xxx属性文件。读→show函数返回数据, 写→store函数接收数据。是设备参数配置和状态查询的标准方式。

在/sys下导出设备属性(推荐方式):

```
static ssize_t status_show(struct device *dev, struct device_attribute *attr,
                           char *buf) {
    return sprintf(buf, "%d\n", get_status());
}

static ssize_t config_store(struct device *dev, struct device_attribute *attr,
                            const char *buf, size_t count) {
    int val;
    if (kstrtoint(buf, 10, &val))
        return -EINVAL;
    set_config(val);
    return count;
}

static DEVICE_ATTR_RO(status); // 只读
static DEVICE_ATTR_RW(config); // 读写

static struct attribute *my_attrs[] = {
    &dev_attr_status.attr,
    &dev_attr_config.attr,
    NULL,
};
ATTRIBUTE_GROUPS(my);
```

Q29: 阻塞与非阻塞IO的驱动实现?

 **秒懂**: 阻塞IO: 数据没准备好时任务睡眠(wait_event)→数据就绪时唤醒(wake_up)。非阻塞IO: 检查 O_NONBLOCK标志→无数据时返回-EAGAIN。两种模式由用户通过open标志选择。

驱动需同时支持阻塞和非阻塞读写:

```
static ssize_t my_read(struct file *filp, char __user *buf,
                      size_t count, loff_t *pos) {
    if (!data_available()) {
        if (filp->f_flags & O_NONBLOCK)
            return -EAGAIN; // 非阻塞:立即返回
        // 阻塞:等待数据
        if (wait_event_interruptible(my_wq, data_available()))
            return -ERESTARTSYS; // 被信号中断
    }
    // 数据就绪,返回数据
    return copy_data_to_user(buf, count);
}
```

Q30: mmap在驱动中的实现?

 **秒懂：** 驱动实现mmap：分配连续物理内存→在mmap回调中remap_pfn_range建立页表映射→用户空间直接访问硬件内存(如帧缓冲区/DMA缓冲)。零拷贝数据传输的关键。

将设备内存/DMA缓冲区映射到用户空间(零拷贝):

```
static int my_mmap(struct file *filp, struct vm_area_struct *vma) {
    unsigned long size = vma->vm_end - vma->vm_start;
    unsigned long pfn = virt_to_phys(my_buffer) >> PAGE_SHIFT;

    // 设置不可缓存(硬件寄存器)
    vma->vm_page_prot = pgprot_noncached(vma->vm_page_prot);

    if (remap_pfn_range(vma, vma->vm_start, pfn, size, vma->vm_page_prot))
        return -EAGAIN;
    return 0;
}
```

 **面试追问：** 上半部和下半部分别处理什么？tasklet/workqueue/threaded_irq怎么选？

 **嵌入式建议：** 上半部:快速确认中断+清标志;下半部:处理数据。推荐threaded_irq(可睡眠,最简洁)。

Q31: misc设备驱动(简化注册)?

 **秒懂：** misc设备是主设备号10的特殊字符设备——仅需misc_register一步即可注册(自动分配次设备号、创建设备节点)。比标准字符设备框架简单很多，适合简单的设备驱动。

misc设备是字符设备的简化版本(主设备号固定为10):

```
#include <linux/miscdevice.h>

static struct file_operations my_fops = {
    .owner = THIS_MODULE,
    .read = my_read,
    .write = my_write,
    .unlocked_ioctl = my_ioctl,
};
```

```
static struct miscdevice my_misc = {
    .minor = MISC_DYNAMIC_MINOR,
    .name = "mymisc",
    .fops = &my_fops,
};
```

// 注册:一步到位(自动创建/dev/mymisc)

```
misc_register(&my_misc);
misc_deregister(&my_misc);
```

Q32: 多个设备实例的管理?

 **秒懂:** 一个驱动管理多个设备实例: 在open中通过inode->i_cdev用container_of找到设备私有数据结构→存到file->private_data。后续read/write/ioctl通过private_data区分不同设备。

一个驱动管理多个相同硬件的设计:

```
struct my_device {
    int id;
    struct cdev cdev;
    struct device *dev;
    void __iomem *regs;
    spinlock_t lock;
    // ... 设备私有数据
};

// open时通过inode找到设备实例
static int my_open(struct inode *inode, struct file *filp) {
    struct my_device *mydev = container_of(inode->i_cdev, struct my_device, cdev);
    filp->private_data = mydev; // 存入file私有数据
    return 0;
}

// read/write/ioctl通过filp->private_data获取设备
static ssize_t my_read(struct file *filp, ...) {
    struct my_device *mydev = filp->private_data;
    // 针对具体设备实例操作
}
```

三、Platform驱动与设备树 (Q33~Q40)

Q33: Platform总线模型是什么?

 **秒懂:** Platform总线是Linux为片上外设(无法枚举的设备)设计的虚拟总线。设备信息(来自设备树)和驱动代码分离→内核匹配后自动调用probe。是嵌入式Linux驱动的核心框架。

Platform总线是Linux中连接设备和驱动的虚拟总线(SoC片上外设):



🧠 秒懂：定义platform_driver结构

现代platform驱动实现:

Q35: 设备树(Device Tree)基础语法?

 **秒懂：** 设备树(.dts)描述硬件：节点(设备)、属性(键值对)、引用(&label)。compatible属性用于驱动匹配。reg/interrupts/clocks等标准属性描述硬件资源。

设备树描述硬件信息(板级差异与驱动代码分离):

```
/ {
    compatible = "vendor,board";

    my_device@40000000 {
        compatible = "vendor,my-device"; // 匹配驱动
        reg = <0x40000000 0x1000>;      // 寄存器基地址和大小
        interrupts = <0 32 4>;           // 中断号
        clocks = <&clk_uart>;            // 时钟引用
        clock-names = "uart_clk";
        gpios = <&gpio0 5 GPIO_ACTIVE_LOW>;
        status = "okay";                 // 启用
    };
};
```

Q36: 在驱动probe中解析设备树?

 **秒懂：** probe函数中用of_property_read_u32、of_get_gpio、irq_of_parse_and_map等API解析设备树节点的属性。platform_get_resource获取reg/irq等资源。解析失败要正确处理错误。

从设备树获取硬件信息:

```
static int my_probe(struct platform_device *pdev) {
    struct device *dev = &pdev->dev;
    struct device_node *np = dev->of_node;

    // 获取寄存器地址(从reg属性)
    struct resource *res = platform_get_resource(pdev, IORESOURCE_MEM, 0);
    void __iomem *base = devm_ioremap_resource(dev, res);

    // 获取中断号
    int irq = platform_get_irq(pdev, 0);

    // 读取自定义属性
    u32 freq;
    of_property_read_u32(np, "clock-frequency", &freq);

    const char *name;
    of_property_read_string(np, "label", &name);

    // 获取GPIO
    int gpio = of_get_named_gpio(np, "reset-gpios", 0);

    return 0;
}
```


Q37: devm_xxx资源管理API?

 **秒懂：** devm_xxx(如devm_kzalloc/devm_request_irq)是设备管理版API——设备销毁时自动释放资源，不需要手动在remove中——释放。大幅减少资源泄漏bug。推荐全面使用。

devm_xxx系列函数实现设备生命周期自动资源管理(类似RAII):

```
// 普通方式(需要手动在remove/出错路径释放)
void *buf = kmalloc(1024, GFP_KERNEL);
// ... 如果后续出错需要kfree(buf)

// devm方式(设备移除/probe失败自动释放)
void *buf = devm_kmalloc(dev, 1024, GFP_KERNEL);
// 不需要手动free!

// 常用devm_xxx:
devm_ioremap_resource()    // 映射IO寄存器
devm_request_irq()         // 注册中断
devm_clk_get()             // 获取时钟
devm_gpio_request()        // 申请GPIO
devm_regulator_get()       // 获取电源
```

Q38: 设备树overlay(动态修改设备树)?

 **秒懂：** 设备树overlay允许在运行时动态添加/修改设备树节点(如树莓派的dtoverlay命令)。不需要重新编译整个设备树。适合扩展板、可配置硬件的场景。

运行时动态添加/修改设备树节点:

```
# 编译overlay
dtc -@ -I dts -O dtb -o overlay.dtbo overlay.dts

# 加载overlay(如树莓派)
dtoverlay myoverlay

# /boot/config.txt
dtoverlay=my-spi-device
```

应用场景： 热插拔扩展板、不同配置的硬件变体。

Q39: regmap框架的作用?

 **秒懂：** regmap抽象了I2C/SPI/MMIO的寄存器访问——统一的regmap_read/regmap_write接口。支持寄存器Cache、访问范围检查、自动字节序转换。现代驱动推荐使用regmap而非直接I2C/SPI读写。

regmap统一I2C/SPI/MMIO的寄存器访问接口:

```
#include <linux/regmap.h>

// I2C设备使用regmap
static const struct regmap_config my_regmap_config = {
    .reg_bits = 8,
```

```

        .val_bits = 8,
        .max_register = 0xFF,
    };

    struct regmap *map = devm_regmap_init_i2c(client, &my_regmap_config);

    // 统一的读写API(无论底层是I2C/SPI/MMIO)
    regmap_read(map, REG_STATUS, &val);
    regmap_write(map, REG_CTRL, 0x01);
    regmap_update_bits(map, REG_CFG, MASK, VALUE);

```

Q40: pinctrl和GPIO子系统?

 **秒懂：** pinctrl子系统管理引脚复用和电气配置(上拉/下拉/驱动强度)。GPIO子系统管理通用IO(输入/输出/中断)。设备树中phandle引用pinctrl和GPIO节点，驱动中用gpiod_get等API操作。

引脚复用和GPIO操作的内核框架：

```

// 设备树中声明pinctrl
my_device {
    pinctrl-names = "default", "sleep";
    pinctrl-0 = <&my_pins_default>;
    pinctrl-1 = <&my_pins_sleep>;
};

// 驱动中使用GPIO(新API: gpiod)
#include <linux/gpio/consumer.h>

struct gpio_desc *reset_gpio;
reset_gpio = devm_gpiod_get(dev, "reset", GPIOD_OUT_HIGH);
gpiod_set_value(reset_gpio, 0); // 拉低
msleep(10);
gpiod_set_value(reset_gpio, 1); // 拉高

```

四、中断处理 (Q41~Q50)

Q41: Linux中断处理框架?

 **秒懂：** request_irq注册中断→ISR执行上半部(irqreturn_t返回值)→下半部(tasklet/workqueue/threaded_irq)处理复杂逻辑。上半部要快(不睡眠)，下半部可以做耗时操作。

Linux中断处理分为顶半部和底半部：

```

#include <linux/interrupt.h>

// 中断处理函数(顶半部：快速，关中断)
static irqreturn_t my_isr(int irq, void *dev_id) {
    struct my_device *dev = dev_id;
    u32 status = readl(dev->regs + INT_STATUS);

    if (!(status & MY_INT_MASK))

```

```

        return IRQ_NONE; // 不是我的中断

// 清中断
write1(status, dev->regs + INT_CLEAR);

// 调度底半部(耗时工作)
tasklet_schedule(&dev->tasklet);
// 或 schedule_work(&dev->work);

return IRQ_HANDLED;
}

// 注册中断
devm_request_irq(dev, irq, my_isr, IRQF_SHARED, "mydev", mydev);

```

Q42: 中断上下文的限制?

 **秒懂**: 中断上下文(ISR)中不能: 睡眠、调用可能睡眠的函数(mutex_lock/kmalloc(GFP_KERNEL))、长时间占用CPU。只能用自旋锁和GFP_ATOMIC分配。原则: 上半部越短越好。

中断上下文中的禁忌(面试必知):

中断上下文中不能做的事:

- X 睡眠/调度(schedule/msleep/mutex_lock)
- X 分配GFP_KERNEL内存(可能触发换页)
- X 调用copy_to/from_user(用户空间可能换出)
- X 获取mutex/semaphore

中断上下文中可以做的事:

- ✓ spinlock(spin_lock_irqsave)
- ✓ 读写硬件寄存器
- ✓ kmalloc(GFP_ATOMIC)
- ✓ 操作内核数据结构
- ✓ 唤醒等待队列(wake_up)
- ✓ schedule_work/tasklet_schedule

Q43: threaded_irq(线程化中断)?

 **秒懂**: threaded_irq: 先执行快速的硬中断处理(上半部)→然后在独立内核线程中执行完整处理(可以睡眠)。request_threaded_irq比tasklet更灵活——内核线程可以调度、可以睡眠。

线程化中断将处理放在内核线程中(可睡眠):

```

// request_threaded_irq: 硬中断仅做确认,线程中处理
static irqreturn_t my_hard_isr(int irq, void *dev_id) {
    // 快速确认中断(顶半部)
    return IRQ_WAKE_THREAD; // 唤醒线程
}

static irqreturn_t my_thread_fn(int irq, void *dev_id) {
    // 线程上下文(可睡眠!)
    mutex_lock(&dev->lock);

```

```

    i2c_smbus_read_byte(dev->client); // 可以做I2C操作
    mutex_unlock(&dev->lock);
    return IRQ_HANDLED;
}

devm_request_threaded_irq(dev, irq, my_hard_isr, my_thread_fn,
                          IRQF_ONESHOT, "mydev", mydev);

```

Q44: tasklet和softirq的区别?

 **秒懂：** tasklet运行在软中断上下文(不能睡眠, 同CPU串行)。softirq是内核的底层机制(网络/定时器等核心用)。实际驱动中: 简单下半部tasklet→需要睡眠用workqueue→最灵活用threaded_irq。

底半部实现方式的选择:

机制	执行上下文	并发	适用
softirq	软中断	可多CPU并行	高频(网络/块设备)
tasklet	软中断	同一tasklet不并发	普通驱动
workqueue	内核线程	可睡眠	需要阻塞的操作
threaded_irq	内核线程	可睡眠	现代驱动首选

Q45: 中断共享(IRQF_SHARED)?

 **秒懂：** 多个设备共享同一中断线: request_irq传IRQF_SHARED→ISR中先检查自己设备是否产生了中断(读状态寄存器)→是则处理并返回IRQ_HANDLED, 否则返回IRQ_NONE。

多个设备共享同一中断线(PCI常见):

```

// 共享中断注册
request_irq(irq, my_isr, IRQF_SHARED, "mydev", my_private_data);

// ISR中必须判断是否是自己的中断
static irqreturn_t my_isr(int irq, void *dev_id) {
    struct my_device *dev = dev_id;
    if (!(readl(dev->regs + STATUS) & INT_PENDING))
        return IRQ_NONE; // 不是我的, 让下一个处理
    // 是我的中断, 处理...
    return IRQ_HANDLED;
}

```

Q46: 中断的enable/disable?

 **秒懂：** disable_irq禁用某中断(等正在执行的ISR完成), enable_irq使能。local_irq_disable禁用本CPU所有中断。中断禁用时间要尽量短——影响系统响应性。

控制中断的开关:

```

// 全局关中断(危险, 尽量少用)

```

```

local_irq_disable(); // 关当前CPU中断
local_irq_enable();

// 保存/恢复中断状态(正确做法)
unsigned long flags;
local_irq_save(flags); // 关中断+保存状态
// 临界区
local_irq_restore(flags); // 恢复原状态

// 关闭特定中断线
disable_irq(irq); // 同步等待当前ISR执行完
disable_irq_nosync(irq); // 不等待
enable_irq(irq);

```

Q47: 中断亲和性(IRQ Affinity)?

 **秒懂：** irq_set_affinity将中断绑定到特定CPU核：多核系统中让特定中断只由指定核处理。减少Cache失效，改善实时性。嵌入式多核SoC(如Cortex-A多核)中常用。

指定中断由哪个CPU处理：

```

# 查看中断分布
cat /proc/interrupts

# 设置IRQ绑定到CPU0
echo 1 > /proc/irq/32/smp_affinity # bitmask: CPU0=1, CPU1=2

# 嵌入式优化：关键中断绑定到专用核
# 如：网卡中断绑CPU0，其他绑CPU1

```

Q48: GPIO中断的使用?

 **秒懂：** 设备树中配置GPIO中断：interrupts属性指定→驱动中gpiod_to_irq获取虚拟中断号→request_irq注册。设置触发方式(上升沿/下降沿/双边沿)。

GPIO作为外部中断源(按键/传感器就绪)：

```

int irq = gpiod_to_irq(my_gpio); // GPIO转IRQ号

devm_request_irq(dev, irq, button_isr,
                 IRQF_TRIGGER_FALLING | IRQF_TRIGGER_RISING,
                 "button", dev);

static irqreturn_t button_isr(int irq, void *dev_id) {
    int val = gpiod_get_value(my_gpio);
    printk("Button %s\n", val ? "released" : "pressed");
    return IRQ_HANDLED;
}

```

Q49: 中断延迟测量?

 **秒懂**: 在GPIO中断触发时翻转另一个GPIO→示波器测量两者延迟。或用ftrace的irq跟踪。中断延迟=硬件延迟+ISR调度延迟。Linux一般几十微秒, PREEMPT_RT补丁可降到微秒级。

评估中断响应时间(实时性指标):

```
# cyclicttest测试中断延迟
cyclicttest -p 80 -t 1 -n -i 1000 -l 10000
# -p 80: 实时优先级
# -t 1: 1个线程
# -i 1000: 1ms间隔
# 结果: Min/Avg/Max 延迟(us)

# ftrace跟踪中断
echo irq > /sys/kernel/debug/tracing/set_event
cat /sys/kernel/debug/tracing/trace
```

Q50: 中断下半部选择指南?

 **秒懂**: 选择指南: 最轻量→tasklet(不能睡眠)→workqueue(能睡眠)→threaded_irq(最灵活)。简单且快的用tasklet, 需要I2C/SPI通信的用workqueue或threaded_irq。

面试需回答"什么场景用什么底半部":

选择决策树:

需要睡眠(I2C/mutex)?

- └ 是 → workqueue 或 threaded_irq
- └ 否 → 执行时间长?
 - └ 是 → tasklet
 - └ 否 → 直接在ISR中处理

现代驱动推荐:

- 简单操作: 直接在ISR中完成
- 需要睡眠: threaded_irq(最简洁)
- 需要延迟执行: work_struct + schedule_work

五、I2C/SPI子系统 (Q51~Q55)

Q51: I2C子系统架构?

 **秒懂**: I2C子系统分三层: I2C核心(内核框架)→适配器驱动(控制器硬件)→设备驱动(传感器等)。驱动开发者只需写设备驱动层——注册i2c_driver+probe中初始化设备。

Linux I2C子系统分为三层:

I2C设备驱动	i2c_driver (如传感器驱动)
I2C核心层	i2c_transfer/smbus接口
I2C适配器驱动	i2c_adapter (如SoC的I2C控制器驱动)

Q52: I2C设备驱动编写?

 **秒懂:** 定义i2c_driver(.probe/.remove/.id_table/.of_match_table)→module_i2c_driver注册。probe中get i2c_client→i2c_smbus_read_byte_data读寄存器→初始化设备。

使用I2C子系统编写传感器驱动:

```
#include <linux/i2c.h>

static int my_i2c_probe(struct i2c_client *client) {
    dev_info(&client->dev, "I2C device probed at 0x%02x\n", client->addr);

    // 读寄存器
    int val = i2c_smbus_read_byte_data(client, REG_ID);
    if (val != EXPECTED_ID)
        return -ENODEV;

    return 0;
}

static void my_i2c_remove(struct i2c_client *client) {
    dev_info(&client->dev, "Removed\n");
}

static const struct of_device_id my_of_match[] = {
    { .compatible = "vendor,my-sensor" },
    {},
};

static struct i2c_driver my_i2c_driver = {
    .driver = {
        .name = "my-sensor",
        .of_match_table = my_of_match,
    },
    .probe = my_i2c_probe,
    .remove = my_i2c_remove,
};

module_i2c_driver(my_i2c_driver);
```

Q53: SPI驱动编写?

秒懂：定义spi_driver(.probe/.remove/.of_match_table)→module_spi_driver注册。probe中get spi_device→spi_write_then_read或spi_transfer全双工通信→初始化设备。

SPI设备驱动(如Flash/LCD):

```
#include <linux/spi/spi.h>

static int my_spi_probe(struct spi_device *spi) {
    spi->mode = SPI_MODE_0;
    spi->bits_per_word = 8;
    spi->max_speed_hz = 10000000; // 10MHz
    spi_setup(spi);

    // SPI传输
    uint8_t tx[] = {0x9F}; // Read JEDEC ID
    uint8_t rx[4] = {0};
    struct spi_transfer xfer = {
        .tx_buf = tx,
        .rx_buf = rx,
        .len = sizeof(tx),
    };
    struct spi_message msg;
    spi_message_init(&msg);
    spi_message_add_tail(&xfer, &msg);
    spi_sync(spi, &msg);

    return 0;
}

static struct spi_driver my_spi_driver = {
    .driver = { .name = "my-flash", .of_match_table = my_of_match },
    .probe = my_spi_probe,
    .remove = my_spi_remove,
};
module_spi_driver(my_spi_driver);
```

Q54: I2C和SPI的对比(驱动视角)?

秒懂：I2C：两线、多从机(地址区分)、速度低(400K)、适合少量低速设备。SPI：四线、CS片选、速度高(几十M)、适合高速数据传输。驱动视角：I2C用regmap更简洁，SPI注意全双工。

从驱动开发角度的差异：

对比项	I2C	SPI
总线线数	2(SDA+SCL)	4+(MOSI/MISO/CLK/CS)
地址	7位设备地址	CS片选(无地址)
速率	100K/400K/3.4M	几十MHz

对比项	I2C	SPI
驱动框架	i2c_driver + i2c_client	spi_driver + spi_device
传输API	i2c_smbus_xxx / i2c_transfer	spi_sync / spi_async
全双工	否(半双工)	是

Q55: DMA在驱动中的使用？

 **秒懂：** DMA在驱动中：dma_alloc_coherent分配DMA缓冲区→配置DMA通道(源/目标/长度/方向)→启动传输→等待完成中断。注意Cache一致性(使用DMA一致性映射或手动同步)。

DMA实现数据零拷贝传输(CPU不参与搬运)：

```
#include <linux/dma-mapping.h>

// 一致性DMA映射(驱动和设备共享的buffer)
void *vaddr = dma_alloc_coherent(dev, size, &dma_addr, GFP_KERNEL);
// vaddr: CPU虚拟地址
// dma_addr: 设备DMA地址

// 告诉硬件DMA地址
writel(dma_addr, dev->regs + DMA_ADDR_REG);
writel(size, dev->regs + DMA_LEN_REG);
writel(DMA_START, dev->regs + DMA_CTRL_REG);

// 等待DMA完成(中断)
wait_for_completion(&dev->dma_done);

// 释放
dma_free_coherent(dev, size, vaddr, dma_addr);
```

六、电源管理与时钟（Q56~Q60）

Q56: Linux电源管理框架(PM)?

 **秒懂：** PM框架：设备驱动实现dev_pm_ops(suspend/.resume)回调。系统进入挂起时内核依次调用所有设备的suspend(保存状态/停止操作)→唤醒时调用resume(恢复状态)。

驱动需要实现suspend/resume以支持系统睡眠：

```
static int my_suspend(struct device *dev) {
    // 保存状态、关闭硬件
    save_regs(mydev);
    clk_disable(mydev->clk);
    return 0;
}

static int my_resume(struct device *dev) {
    // 恢复硬件状态
```

```

    clk_enable(mydev->clk);
    restore_regs(mydev);
    return 0;
}

static SIMPLE_DEV_PM_OPS(my_pm_ops, my_suspend, my_resume);

static struct platform_driver my_driver = {
    .driver = {
        .pm = &my_pm_ops,
    },
};

```

Q57: Runtime PM(运行时电源管理)?

 **秒懂：** Runtime PM让设备在不使用时自动idle/suspend(不等系统挂起)。pm_runtime_get_sync(使用前激活)→pm_runtime_put_autosuspend(用完后延时挂起)。最大化省电。

Runtime PM在设备空闲时自动关闭：

```

#include <linux/pm_runtime.h>

// probe中启用
pm_runtime_enable(dev);
pm_runtime_set_autosuspend_delay(dev, 200); // 200ms空闲后挂起
pm_runtime_use_autosuspend(dev);

// 使用设备前
pm_runtime_get_sync(dev); // 唤醒设备
// 操作硬件...
pm_runtime_put_autosuspend(dev); // 标记空闲(延迟挂起)

// 实现runtime callbacks
static int my_runtime_suspend(struct device *dev) {
    clk_disable(mydev->clk);
    return 0;
}
static int my_runtime_resume(struct device *dev) {
    clk_enable(mydev->clk);
    return 0;
}

```

Q58: Clock框架(CCF)使用?

 **秒懂：** CCF(Common Clock Framework)统一管理设备时钟：clk_get获取时钟→clk_prepare_enable使能→clk_set_rate设频率→clk_disable_unprepare禁用。设备树中clocks属性引用时钟源。

时钟控制框架管理SoC中的各种时钟：

```

#include <linux/clock.h>

// 获取时钟

```

```

struct clk *clk = devm_clk_get(dev, "uart_clk");

// 设置频率
clk_set_rate(clk, 48000000); // 48MHz

// 使能/关闭
clk_prepare_enable(clk);
// 使用中...
clk_disable_unprepare(clk);

// 获取当前频率
unsigned long rate = clk_get_rate(clk);

```

Q59: Regulator(电源)框架?

 **秒懂：** Regulator框架管理电源域：regulator_get获取→regulator_enable使能→regulator_set_voltage设电压→regulator_disable禁用。设备树中描述电源供应关系。

管理设备的供电电源：

```

#include <linux/regulator/consumer.h>

struct regulator *vdd = devm_regulator_get(dev, "vdd");

// 设置电压
regulator_set_voltage(vdd, 3300000, 3300000); // 3.3V

// 使能电源
regulator_enable(vdd);
// 使用设备...
regulator_disable(vdd);

```

Q60: Devicetree中的电源和时钟描述?

 **秒懂：** 设备树中clocks引用时钟源(clock-names给别名), power-domains引用电源域。of_clk_get_by_name解析时钟, devm_regulator_get解析电源。硬件资源描述和代码分离。

设备树中描述硬件的时钟和电源依赖：

```

my_device@4000 {
    compatible = "vendor,mydev";
    reg = <0x4000 0x100>;

    // 时钟
    clocks = <&rcc UART1_CLK>;
    clock-names = "uart_clk";

    // 电源
    vdd-supply = <&reg_3v3>;

    // 复位
    resets = <&rcc UART1_RST>;

```

```
reset-names = "uart_rst";  
};
```

七、驱动调试（Q61~Q65）

Q61: 内核Oops信息解读？

 **秒懂：** Oops是内核遇到异常时的错误信息：寄存器快照+调用栈(backtrace)+出错位置(PC)。用addr2line或gdb将地址翻译成源码行。Oops后系统可能继续运行但不可靠。

内核oops是驱动最常见的崩溃形式(类似用户态的段错误)：

Oops典型信息：

```
unable to handle kernel NULL pointer dereference at virtual address 0x00000010  
PC is at my_driver_read+0x28/0x100 [my_module]  
LR is at vfs_read+0x88/0x1a0
```

Call trace:

```
my_driver_read+0x28/0x100  
vfs_read+0x88/0x1a0  
sys_read+0x44/0x90
```

分析步骤：

1. 看错误类型：NULL pointer dereference
2. 看PC位置：my_driver_read+0x28（偏移0x28）
3. 用addr2line或objdump定位源码行：

```
arm-linux-gnueabi-hf-addr2line -e my_module.ko 0x28 -f
```

Q62: ftrace跟踪内核函数？

 **秒懂：** ftrace动态跟踪内核函数：echo function > current_tracer→设置filter→cat trace查看。可以看到函数调用链和耗时。function_graph跟踪器能看到调用树和时间。

ftrace是内核内置的跟踪框架：

```
# 跟踪函数调用  
echo function > /sys/kernel/debug/tracing/current_tracer  
echo my_driver_* > /sys/kernel/debug/tracing/set_ftrace_filter  
echo 1 > /sys/kernel/debug/tracing/tracing_on  
# 操作设备...  
cat /sys/kernel/debug/tracing/trace  
  
# 跟踪函数执行时间  
echo function_graph > /sys/kernel/debug/tracing/current_tracer  
  
# 跟踪事件  
echo 1 > /sys/kernel/debug/tracing/events/irq/enable  
echo 1 > /sys/kernel/debug/tracing/events/sched/enable
```

Q63: KGDB内核调试？

 **秒懂：** KGDB通过串口或网络连接GDB到内核——设断点、单步、查看内核变量。配置内核开启KGDB→启动参数指定调试串口→GDB连接。用于复杂内核问题的最终调试手段。

通过串口使用GDB调试内核代码：

```
# 目标板内核启动参数
kgdboc=ttyS0,115200 kgdbwait

# 主机端GDB
arm-linux-gnueabi-gdb vmlinux
(gdb) target remote /dev/ttyUSB0
(gdb) break my_driver_probe
(gdb) continue
```

全网最详细的嵌入式实习/秋招/春招公司清单，正在更新中，需要可关注微信公众号：嵌入式校招菌。

Q64: devmem直接读写寄存器？

 **秒懂：** devmem/devmem2直接在用户空间读写物理地址(通过/dev/mem mmap)。devmem 0x40021000 查看寄存器值。快速确认硬件状态的利器,但有安全风险(需root)。

调试阶段直接从用户态访问硬件寄存器：

```
# devmem2(或busybox devmem)
devmem 0x40000000 # 读
devmem 0x40000000 32 0x1 # 写(32位值0x1)

# 或通过/dev/mem
dd if=/dev/mem bs=4 count=1 skip=$((0x40000000/4)) | xxd
```

Q65: 驱动开发常见bug和排查？

 **秒懂：** 常见bug：忘记释放资源(内存/中断/GPIO)→用devm_xxx自动管理。忘记处理并发(多个进程同时操作)→加锁。忘记检查返回值→每个API都要检查错误。中断中睡眠→崩溃。

驱动开发中的典型错误：

问题	症状	排查方法
空指针	Oops + NULL deref	检查probe返回值
竞态	偶发数据错误	lockdep检测/review锁
中断中睡眠	BUG: scheduling while atomic	检查ISR中是否有mutex/kmalloc(GFP_KERNEL)
内存泄漏	kmemleak报告	使用devm_xxx
死锁	系统挂起	lockdep/sysrq-t

```
# 开启lockdep(编译内核时CONFIG_PROVE_LOCKING=y)
# 开启kmemleak
echo scan > /sys/kernel/debug/kmemleak
cat /sys/kernel/debug/kmemleak
```

八、高级驱动主题（Q66~Q75）

Q66: 块设备驱动基础？

 **秒懂：** 块设备驱动处理以块为单位的IO：注册gendisk→实现request/bio处理函数→IO调度器排队优化。比字符设备复杂——要处理IO请求队列和缓冲。eMMC/SD/NAND的底层。

块设备以固定大小块(512B/4KB)为单位读写：

```
#include <linux/blkdev.h>

static struct gendisk *my_disk;
static struct request_queue *my_queue;

static void my_request(struct request_queue *q) {
    struct request *req;
    while ((req = blk_fetch_request(q)) != NULL) {
        // 处理请求
        if (rq_data_dir(req) == READ)
            read_from_device(req);
        else
            write_to_device(req);
        __blk_end_request_all(req, 0);
    }
}
```

Q67: 网络设备驱动基础？

 **秒懂：** 网络设备驱动实现net_device_ops(ndo_start_xmit发送/.ndo_open启用等)。收到数据包时调用netif_rx放入协议栈。不像字符设备有/dev节点——通过ifconfig/ip命令操作。

网络设备驱动通过net_device结构体注册：

```
#include <linux/netdevice.h>

static int my_net_open(struct net_device *dev) {
    netif_start_queue(dev);
    return 0;
}

static netdev_tx_t my_net_xmit(struct sk_buff *skb, struct net_device *dev) {
    // 发送数据包到硬件
    send_to_hw(skb->data, skb->len);
    dev_kfree_skb(skb);
    return NETDEV_TX_OK;
}
```

```

}

// 收到数据包(中断中调用)
void my_net_rx(struct net_device *dev, void *data, int len) {
    struct sk_buff *skb = netdev_alloc_skb(dev, len);
    memcpy(skb_put(skb, len), data, len);
    skb->protocol = eth_type_trans(skb, dev);
    netif_rx(skb); // 递交上层协议栈
}

```

Q68: Input子系统驱动?

 **秒懂：** Input子系统驱动报告输入事件：input_allocate_device→设置支持的事件类型
→input_register_device→input_report_key/input_report_abs报告事件。按键/触摸屏/传感器都走这个框架。

按键/触摸屏等输入设备驱动框架：

```

#include <linux/input.h>

struct input_dev *input = devm_input_allocate_device(dev);
input->name = "my-buttons";
input->evbit[0] = BIT_MASK(EV_KEY);
input_set_capability(input, EV_KEY, KEY_POWER);
input_register_device(input);

// 上报事件(中断中)
input_report_key(input, KEY_POWER, 1); // 按下
input_sync(input);
input_report_key(input, KEY_POWER, 0); // 释放
input_sync(input);

```

Q69: IIO子系统(传感器驱动)?

 **秒懂：** IIO(Industrial IO)是传感器驱动的标准框架：ADC、加速度计、温湿度传感器等。提供统一的sysfs接口(in_voltage0_raw)和缓冲区采集(triggered buffer)。比自己写字符设备规范。

Industrial I/O子系统用于ADC/DAC/IMU等模拟传感器：

```

#include <linux/iio/iio.h>

static const struct iio_chan_spec my_channels[] = {
    {
        .type = IIO_TEMP,
        .info_mask_separate = BIT(IIO_CHAN_INFO_RAW) | BIT(IIO_CHAN_INFO_SCALE),
    },
};

static int my_read_raw(struct iio_dev *indio_dev,
                      struct iio_chan_spec const *chan,
                      int *val1, int *val2, long mask) {
    switch (mask) {
        case IIO_CHAN_INFO_RAW:

```

```

        *val = read_adc_value();
        return IIO_VAL_INT;
    case IIO_CHAN_INFO_SCALE:
        *val = 0;
        *val2 = 100000; // 0.1 度/LSB
        return IIO_VAL_INT_PLUS_MICRO;
    }
    return -EINVAL;
}

```

Q70: Framebuffer驱动(LCD)?

 **秒懂：**Framebuffer驱动：注册fb_info→实现fb_ops(fb_fillrect画矩形/fb_setpar设参数等)→mmap让用户空间直接写帧缓冲。LCD/OLED显示驱动的传统方式(现在用DRM框架更多)。

简单的显示设备驱动：

```

#include <linux/fb.h>

static struct fb_info *info;
info = framebuffer_alloc(sizeof(struct my_fb), dev);
info->fix.smem_start = phys_addr; // 显存物理地址
info->fix.smem_len = width * height * 4; // 显存大小
info->var.xres = 800;
info->var.yres = 480;
info->var.bits_per_pixel = 32;
info->screen_base = ioremap(phys_addr, info->fix.smem_len);
register_framebuffer(info);

```

Q71: V4L2视频驱动基础?

 **秒懂：**V4L2(Video for Linux 2)是视频设备的标准框架：注册video_device→实现v4l2_ioctl_ops→管理视频缓冲区(videobuf2)。摄像头和视频编解码器驱动都用V4L2。

Video4Linux2摄像头驱动框架：

```

// V4L2驱动需要实现的核心操作：
// 1. 查询能力(VIDIOC_QUERYCAP)
// 2. 设置格式(VIDIOC_S_FMT)
// 3. 请求缓冲区(VIDIOC_REQBUFS)
// 4. 队列管理(QBUF/DQBUF)
// 5. 开始/停止采集(STREAMON/STREAMOFF)

static const struct v4l2_ioctl_ops my_ioctl_ops = {
    .vidioc_querycap = my_querycap,
    .vidioc_s_fmt_vid_cap = my_s_fmt,
    .vidioc_reqbufs = vb2_ioctl_reqbufs,
    .vidioc_streamon = vb2_ioctl_streamon,
    // ...
};

```


Q72: USB设备驱动?

💡 秒懂: USB设备驱动: 定义usb_driver(.probe/.disconnect/.id_table匹配VID/PID)→usb_register注册。probe中获取usb_interface→分析端点→设置URB进行数据传输。

USB驱动通过VID/PID匹配设备:

```
/* USB设备驱动? - 示例实现 */
#include <linux/usb.h>

static const struct usb_device_id my_usb_id[] = {
    { USB_DEVICE(0x1234, 0x5678) },
    {},
};
MODULE_DEVICE_TABLE(usb, my_usb_id);

static int my_usb_probe(struct usb_interface *intf,
                        const struct usb_device_id *id) {
    struct usb_device *udev = interface_to_usbdev(intf);
    dev_info(&intf->dev, "USB device %04x:%04x connected\n",
             id->idVendor, id->idProduct);
    return 0;
}

static struct usb_driver my_usb_driver = {
    .name = "my_usb",
    .probe = my_usb_probe,
    .disconnect = my_usb_disconnect,
    .id_table = my_usb_id,
};
module_usb_driver(my_usb_driver);
```

Q73: 内核并发控制实践?

💡 秒懂: 内核并发控制实践: 资源只从中断访问→spin_lock_irqsave。进程上下文且可能睡眠→mutex。读多写少→RCU或rwlock。简单计数器→atomic_t。Per-CPU数据→无需锁。

驱动中的并发场景和保护策略:

```
// 场景1: 多进程同时open/read设备
static DEFINE_MUTEX(dev_mutex);
static int my_open(struct inode *inode, struct file *filp) {
    mutex_lock(&dev_mutex);
    // 独占操作
    mutex_unlock(&dev_mutex);
    return 0;
}

// 场景2: ISR和进程共享数据
static DEFINE_SPINLOCK(data_lock);
// ISR中:
spin_lock(&data_lock);
```

```
update_shared_data();
spin_unlock(&data_lock);
// 进程中:
spin_lock_irqsave(&data_lock, flags);
read_shared_data();
spin_unlock_irqrestore(&data_lock, flags);
```

Q74: 热插拔和设备模型?

 **秒懂**: 设备热插拔: 内核发现设备→创建device→匹配driver→调用probe。拔出时→调用remove。设备模型(bus/device/driver三角关系)是Linux驱动框架的核心设计思想。

Linux设备模型(bus/device/driver)支持热插拔:

设备模型核心:

kobject → 目录(/sys)
kset → 容器
bus_type → 总线(匹配device和driver)
device → 硬件实例
device_driver → 驱动代码

热插拔流程(USB为例):

1. 物理连接变化 → USB主控检测
2. 内核创建usb_device → 加入USB总线
3. USB总线遍历已注册的usb_driver → VID/PID匹配
4. 匹配成功 → 调用probe()
5. 拔出 → 调用disconnect()

Q75: Buildroot/Yocto下构建驱动?

 **秒懂**: Buildroot: add .mk配置编译模块(简单直接)。Yocto: 写.bb recipe声明源码和编译步骤(灵活但学习曲线陡)。两者都支持交叉编译内核模块和打包到rootfs。

嵌入式Linux系统中集成驱动:

```
# Buildroot包示例(package/mydriver/mydriver.mk)
MYDRIVER_VERSION = 1.0
MYDRIVER_SITE = $(TOPDIR)/../mydriver
MYDRIVER_SITE_METHOD = local

define MYDRIVER_BUILD_CMDS
    $(MAKE) -C $(LINUX_DIR) M=$(@D) ARCH=$(KERNEL_ARCH) \
        CROSS_COMPILE=$(TARGET_CROSS) modules
endef

define MYDRIVER_INSTALL_TARGET_CMDS
    $(MAKE) -C $(LINUX_DIR) M=$(@D) ARCH=$(KERNEL_ARCH) \
        INSTALL_MOD_PATH=$(TARGET_DIR) modules_install
endef

$(eval $(generic-package))
```

九、面试高频进阶（Q76~Q80）

Q76: 从probe到设备可用的完整流程？

秒懂：完整流程：设备树匹配compatible→内核调用probe→probe中获取资源(时钟/GPIO/中断/内存)→初始化硬件→注册到子系统(字符设备/input/IIO等)→创建设备节点→用户可访问。

面试必答: 一个platform设备从设备树到用户可操作的全流程：

1. 内核解析设备树 → 创建platform_device
 2. platform_bus匹配compatible → 调用driver.probe()
 3. probe()中：
 - a. devm_ioremap_resource()映射寄存器
 - b. devm_request_irq()注册中断
 - c. devm_clk_get() + clk_prepare_enable()
 - d. alloc_chrdev_region() + cdev_add() 注册字符设备
 - e. class_create() + device_create() → /dev节点出现
 4. 用户open("/dev/mydev") → 调用fops.open
 5. 用户read/write/ioctl → 调用对应fops函数

Q77: 内核模块和用户程序的区别？

秒懂：内核模块：运行在内核空间(ring 0)、不能用标准C库(用printk不用printf)、崩溃会导致整个系统宕机、用kmalloc不用malloc。用户程序反之。两者API完全不同。

面试常问"内核编程和应用编程有什么不同"：

对比项	内核模块	用户程序
运行空间	内核态(Ring0)	用户态(Ring3)
内存	kmalloc/kzalloc	malloc
错误后果	内核oops/系统崩溃	段错误/进程终止
调试	printk/fttrace/KGDB	gdb/printf
库函数	不能用libc(无printf/malloc)	自由使用
浮点	不建议使用	正常使用
栈大小	~8KB(很小!)	~8MB
并发	必须保护(中断/多核)	可选

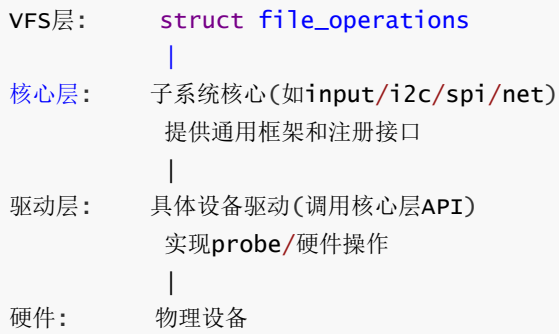
Q78: Linux驱动的分层设计思想？

秒懂：Linux驱动分层：①设备层(设备树描述硬件) ②总线层(platform/I2C/SPI匹配) ③驱动层(probe初始化) ④子系统层(framebuffer/input/IIO等框架)。分层让驱动代码最大化复用。

理解Linux驱动的分层架构(面试加分)：

用户空间： APP (read/write/ioctl)

系统调用边界



分层优势:

- 代码复用(核心层通用逻辑)
- 解耦(换一套硬件只改驱动层)
- 标准化(用户接口统一)

Q79: DMA控制器驱动(DMAEngine)?

 **秒懂:** DMAEngine提供标准的DMA API: dmaengine_prep_slave_sg准备传输描述符
→dmaengine_submit提交→dma_async_issue_pending触发。驱动不直接操作DMA控制器寄存器。

使用Linux DMA Engine API进行DMA传输:

```
#include <linux/dmaengine.h>

struct dma_chan *chan = dma_request_chan(dev, "rx");

// 准备DMA传输
struct dma_async_tx_descriptor *desc;
desc = dmaengine_prep_slave_single(chan, dma_addr, len,
                                   DMA_DEV_TO_MEM, DMA_PREP_INTERRUPT);
desc->callback = dma_complete_callback;
desc->callback_param = mydev;

// 提交并启动
dmaengine_submit(desc);
dma_async_issue_pending(chan);

// 等待完成
wait_for_completion(&mydev->dma_done);
```

Q80: 如何学习和调试一个新的驱动子系统?

 **秒懂:** 学习新驱动子系统的方法: ①读Documentation/下的文档 ②读最简单的现有驱动示例代码
③printk/fttrace跟踪执行流程 ④写最小demo验证理解。从简单到复杂渐进式学习。

面试可能问"拿到一个你不熟悉的驱动子系统怎么入手":

1. 文档: Documentation/目录下对应子系统文档
2. 示例: drivers/目录下找简单的参考驱动
3. 接口: include/linux/下的头文件(API定义)
4. 设备树: Documentation/devicetree/bindings/
5. 调试:
 - printk/dev_xxx添加日志
 - ftrace跟踪函数调用
 - /sys/kernel/debug/下的调试文件
 - 内核源码交叉引用(elixir.bootlin.com)

本文档由公众号：**嵌入式校招菌** 原创整理，欢迎关注获取更多嵌入式校招信息

🔥 **嵌入式校招excel** 全年更新中，实习/秋招/春招都包含，纯人工维护，已支持24/25/26届同学毕业，减少招聘信息差，你没听过的公司只要有嵌入式岗位我都会收集，一次付费永久有效，更有嵌入式微信/飞书交流群；用过的同学都说好！

需要的可以关注公众号：**嵌入式校招菌**，对话框回复：**加群**

★ 面经高频补充题（来源：GitHub面经仓库/牛客讨论区/大厂真题整理）

Q81: 如何写一个最简单的Linux字符设备驱动？

💡 秒懂：最简字符设备驱动：alloc_chrdev_region分配设备号→cdev_init绑定fops→cdev_add注册→class_create+device_create创建节点。fops中实现read/write回调。30行代码搞定。

💡 面试高频 | 现场手写代码题 | 大疆/海康/小米面试常考

面试现场能快速写出的最小驱动框架：

```
#include <linux/module.h>
#include <linux/fs.h>
#include <linux/cdev.h>
#include <linux/uaccess.h>

#define DEV_NAME "mydev"
static dev_t dev_num;
static struct cdev my_cdev;
static struct class *my_class;

static int my_open(struct inode *inode, struct file *file) {
    printk("mydev opened\n");
    return 0;
}

static ssize_t my_read(struct file *file, char __user *buf,
                      size_t count, loff_t *pos) {
    char kbuf[] = "hello from kernel";
    if (copy_to_user(buf, kbuf, sizeof(kbuf)))
        return -EFAULT;
}
```

```

    return sizeof(kbuf);
}

static struct file_operations fops = {
    .owner = THIS_MODULE,
    .open = my_open,
    .read = my_read,
};

static int __init my_init(void) {
    alloc_chrdev_region(&dev_num, 0, 1, DEV_NAME);    // 1.分配设备号
    cdev_init(&my_cdev, &fops);                      // 2.初始化cdev
    cdev_add(&my_cdev, dev_num, 1);                  // 3.注册到内核
    my_class = class_create(THIS_MODULE, DEV_NAME);   // 4.创建class
    device_create(my_class, NULL, dev_num, NULL, DEV_NAME); // 5.创建设备节点
    return 0;
}

static void __exit my_exit(void) {
    device_destroy(my_class, dev_num);
    class_destroy(my_class);
    cdev_del(&my_cdev);
    unregister_chrdev_region(dev_num, 1);
}

module_init(my_init);
module_exit(my_exit);
MODULE_LICENSE("GPL");

```

面试追问:

- "为什么用alloc_chrdev_region不用register_chrdev?" → 动态分配设备号,避免冲突
- "copy_to_user为什么不能用memcpy?" → 要检查用户空间地址合法性,memcpy会内核panic

Q82: 设备树(DTS)中如何描述一个I2C设备?

 **秒懂:** I2C设备节点放在I2C控制器节点下: &i2c1 { sensor@68 { compatible = 'vendor,model'; reg = <0x68>; }; };。reg是I2C地址, compatible用于匹配驱动。

 **面试高频** | 嵌入式Linux驱动岗必考 | 需要现场写dts节点

```
// 在板级dts中描述一个I2C传感器(如BMP280)
&i2c1 {
    status = "okay";
    clock-frequency = <400000>; // 400KHz Fast Mode

    bmp280@76 {
        compatible = "bosch,bmp280"; // 用于匹配驱动
        reg = <0x76>; // I2C从机地址
        interrupt-parent = <&gpio1>;
        interrupts = <5 IRQ_TYPE_EDGE_FALLING>; // GPIO1_5下降沿
    };
};
```

关键属性解释：

- `compatible`: 字符串,驱动通过此属性匹配设备(最重要!)
- `reg`: 设备地址(I2C地址/SPI CS号/寄存器基地址)
- `interrupts`: 中断描述
- `status`: "okay"启用 / "disabled"禁用

面试追问：

- "驱动怎么获取设备树属性?" → `of_property_read_u32()` / `of_property_read_string()`
- "compatible匹配规则?" → 驱动的`of_device_id`表和设备树`compatible`字符串逐一比较

Q83: Linux驱动中的阻塞与非阻塞I/O?

 **秒懂：** 阻塞IO用`wait_event_interruptible`+`wake_up`实现等待-唤醒。非阻塞IO检查`O_NONBLOCK`标志→无数据返回-EAGAIN。驱动必须同时支持两种模式。

 **面试高频** | 驱动开发进阶题 | 等待队列(waitqueue)是核心

```
// 驱动中实现阻塞读(等数据到来)
static DECLARE_WAIT_QUEUE_HEAD(read_wq);
static int data_ready = 0;
static char kbuf[64];

// 中断处理中：数据到来时唤醒
irqreturn_t my_irq(int irq, void *dev) {
    // 读取硬件数据到kbuf...
    data_ready = 1;
    wake_up_interruptible(&read_wq); // 唤醒等待的进程
    return IRQ_HANDLED;
}

// 读函数：阻塞等待数据
static ssize_t my_read(struct file *file, char __user *buf,
                      size_t count, loff_t *pos) {
    // 如果是非阻塞模式且无数据,立即返回
    if ((file->f_flags & O_NONBLOCK) && !data_ready)
```

```
        return -EAGAIN;

// 阻塞等待数据就绪
wait_event_interruptible(read_wq, data_ready);

if (copy_to_user(buf, kbuf, count))
    return -EFAULT;
data_ready = 0;
return count;
}
```

嵌入式建议： 驱动中必须支持非阻塞模式(检查O_NONBLOCK),否则应用层的select/poll/epoll无法正常工作。

本文档由公众号：**嵌入式校招菌** 原创整理，欢迎关注获取更多嵌入式校招信息

📌 **嵌入式校招excel** 全年更新中，实习/秋招/春招都包含，纯人工维护，已支持24/25/26届同学毕业，减少招聘信息差，你没听过的公司只要有嵌入式岗位我都会收集，一次付费永久有效，更有嵌入式微信/飞书交流群；用过的同学都说好！

需要的可以关注公众号：**嵌入式校招菌**，对话框回复：**加群**